

Fundación Universitaria CEIPA

Núcleo

Fundamentos de Software

Actividad 3-2: Taller análisis y aplicación de principios SOLID

Presentado por:

Guillermo León Ríos Franco

Carlos Arturo Molina Tomases

Docente:

Simón Peláez Loaiza

Ingeniería de Sistemas

2026

INTRODUCCIÓN

Los principios SOLID constituyen un conjunto de buenas prácticas de diseño orientado a objetos cuyo propósito es desarrollar software más mantenible, escalable, reutilizable y fácil de evolucionar. En este informe se analiza la clase `GestorTransaccionPrincipal`, identificando las violaciones a los principios Single Responsibility Principle (SRP) y Open/Closed Principle (OCP), así como una propuesta conceptual de mejora.

PASO A. ANÁLISIS DEL PRINCIPIO DE RESPONSABILIDAD ÚNICA (SRP)

Diagnóstico de la violación del SRP

El método `procesar_y_validar_y_reportar()` incumple el principio SRP porque concentra tres responsabilidades distintas en un único método, generando alto acoplamiento y dificultando el mantenimiento del sistema.

- Validación de datos: verifica el monto y el tipo de transacción.
- Lógica de negocio: calcula el impacto según el tipo de transacción.
- Generación del reporte: presenta los resultados en texto plano.

Propuesta de refactorización

Se propone separar las responsabilidades en tres clases conceptuales:

- `ValidadorDatos`: valida la información de entrada.
- `MotorLogicaNegocio`: ejecuta las reglas del negocio y calcula el impacto.
- `GeneradorReporte`: construye y presenta los reportes.

Esta separación mejora la cohesión, reduce el acoplamiento, facilita las pruebas unitarias y permite realizar cambios de manera independiente.

PASO B. PRINCIPIO ABIERTO/CERRADO (OCP)

Análisis del riesgo

Modificar directamente la clase `GestorTransaccionPrincipal` para soportar nuevos formatos de reporte como XML o JSON viola el principio OCP, ya que obliga a cambiar

una clase existente para agregar funcionalidades nuevas. Esto aumenta el riesgo de afectar la validación y la lógica del negocio.

Solución conceptual mediante Herencia y Polimorfismo

Se propone una jerarquía compuesta por ReporteBase, ReporteTexto, ReporteXML y ReporteJSON. La lógica del negocio trabajaría únicamente con ReporteBase, mientras que cada clase derivada implementaría su propio formato. Así, nuevos formatos pueden añadirse creando nuevas clases, sin modificar el código existente.

CONCLUSIONES

La separación de responsabilidades mediante SRP y la extensión mediante herencia y polimorfismo para cumplir OCP permiten construir un sistema más mantenible, escalable y robusto, reduciendo riesgos y facilitando futuras ampliaciones.